

Documentación — Módulo 4: Predicción Lineal Simple

Archivo: `modulo_4_prediccion.s`

1. Explicación del algoritmo implementado

El módulo implementa una **predicción lineal simple** sobre los N datos históricos, especificados mediante la línea de comandos en forma de rango (línea inicial y línea final), de una columna perteneciente a un archivo de formato `.csv`. Se asume que la tasa de cambio promedio observada entre el primer y el último dato se mantendrá constante, y con eso se estima el siguiente valor de la serie.

El algoritmo sigue tres pasos matemáticos (todo en enteros con signo):

1. Diferencia total entre el primer dato (X_{inicial}) y el último (X_{final}):

$$\text{DIF} = X_{\text{final}} - X_{\text{inicial}}$$

2. Cambio promedio por intervalo, distribuyendo la diferencia total entre los $N - 1$ intervalos que separan los N datos:

$$\text{PROMEDIO_CAMBIO} = \text{DIF} / (N - 1)$$

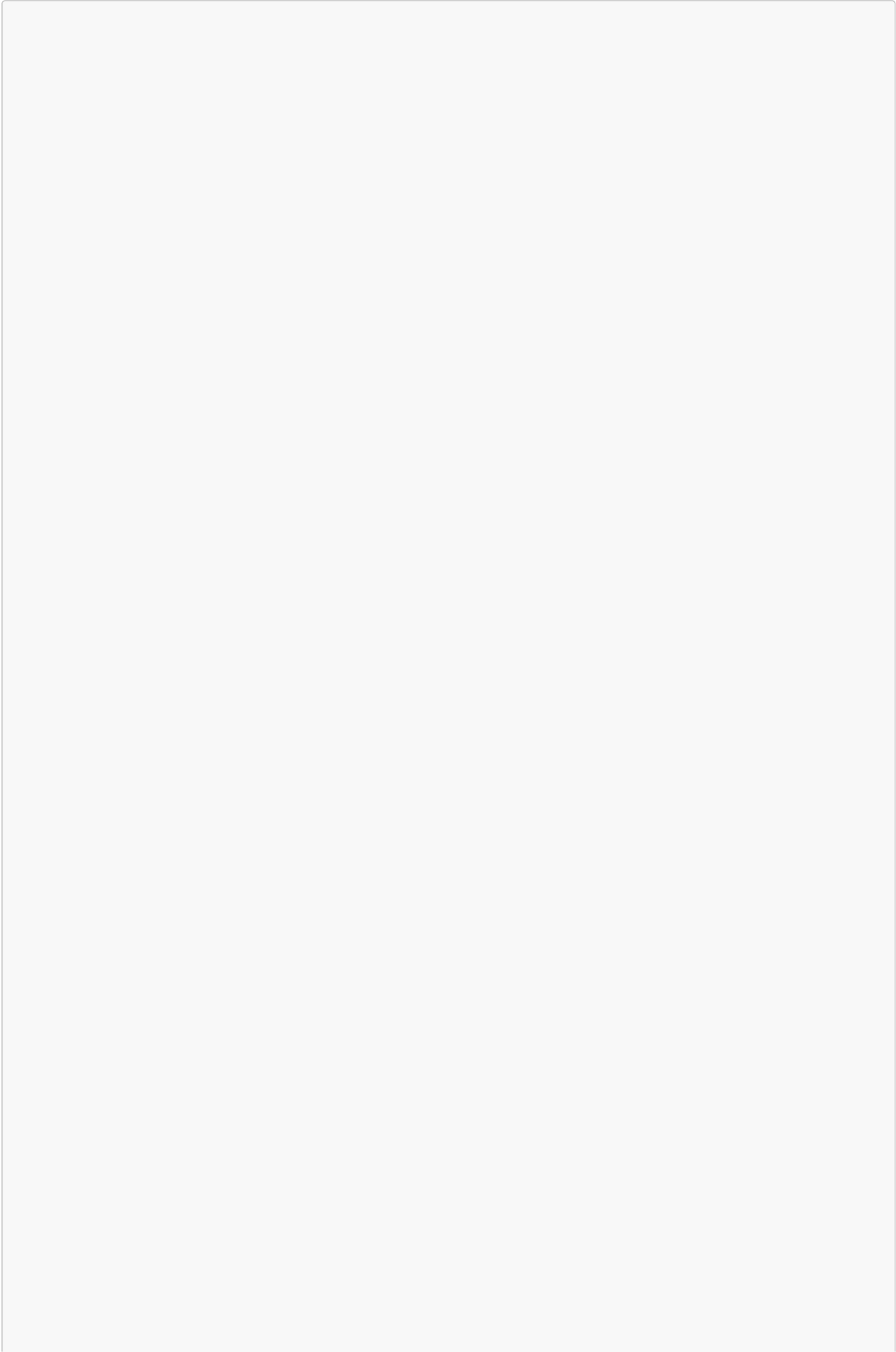
División entera con signo (`sdiv`), el resultado se trunca hacia cero.

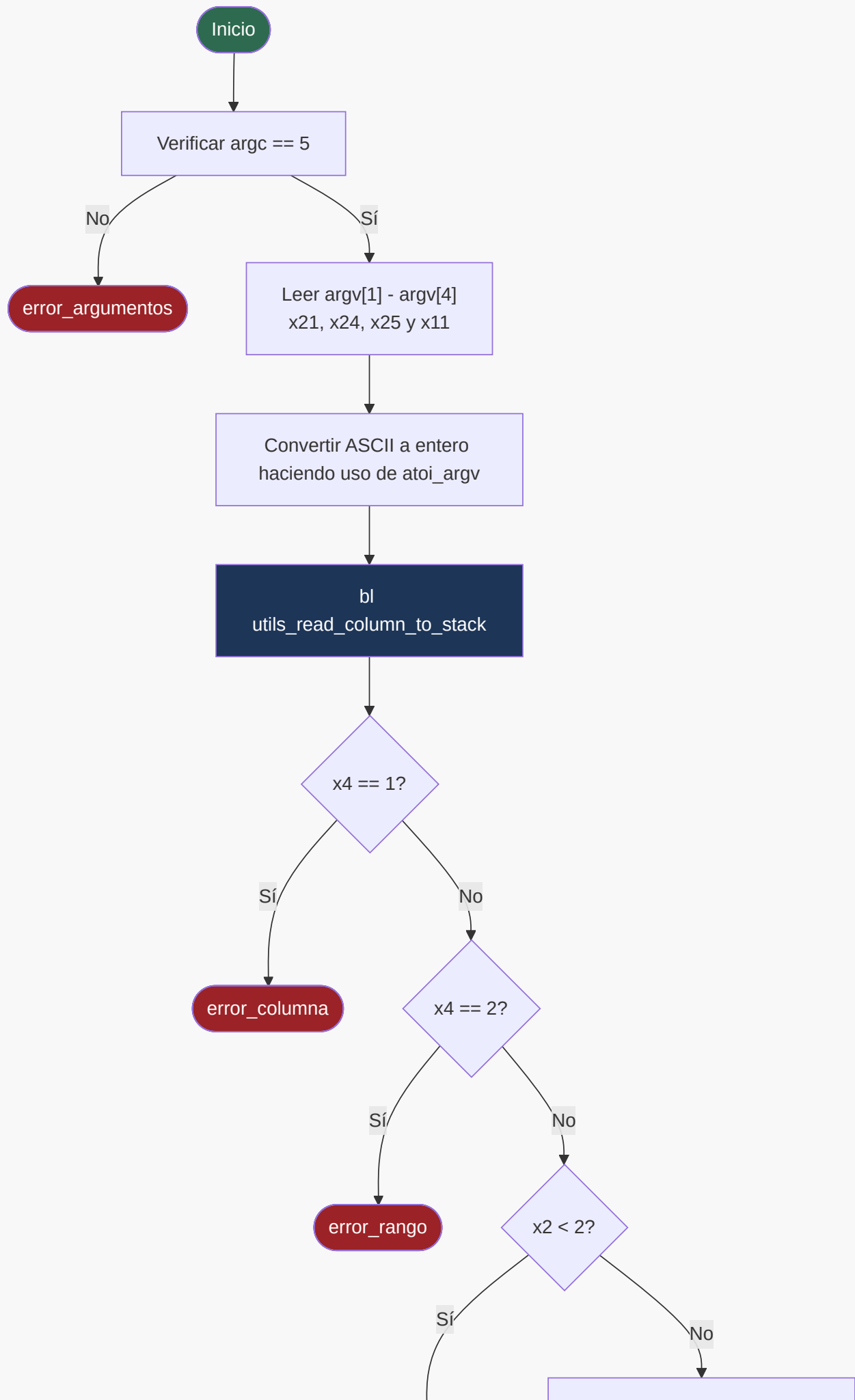
3. Predicción del siguiente valor, asumiendo que la tendencia continúa:

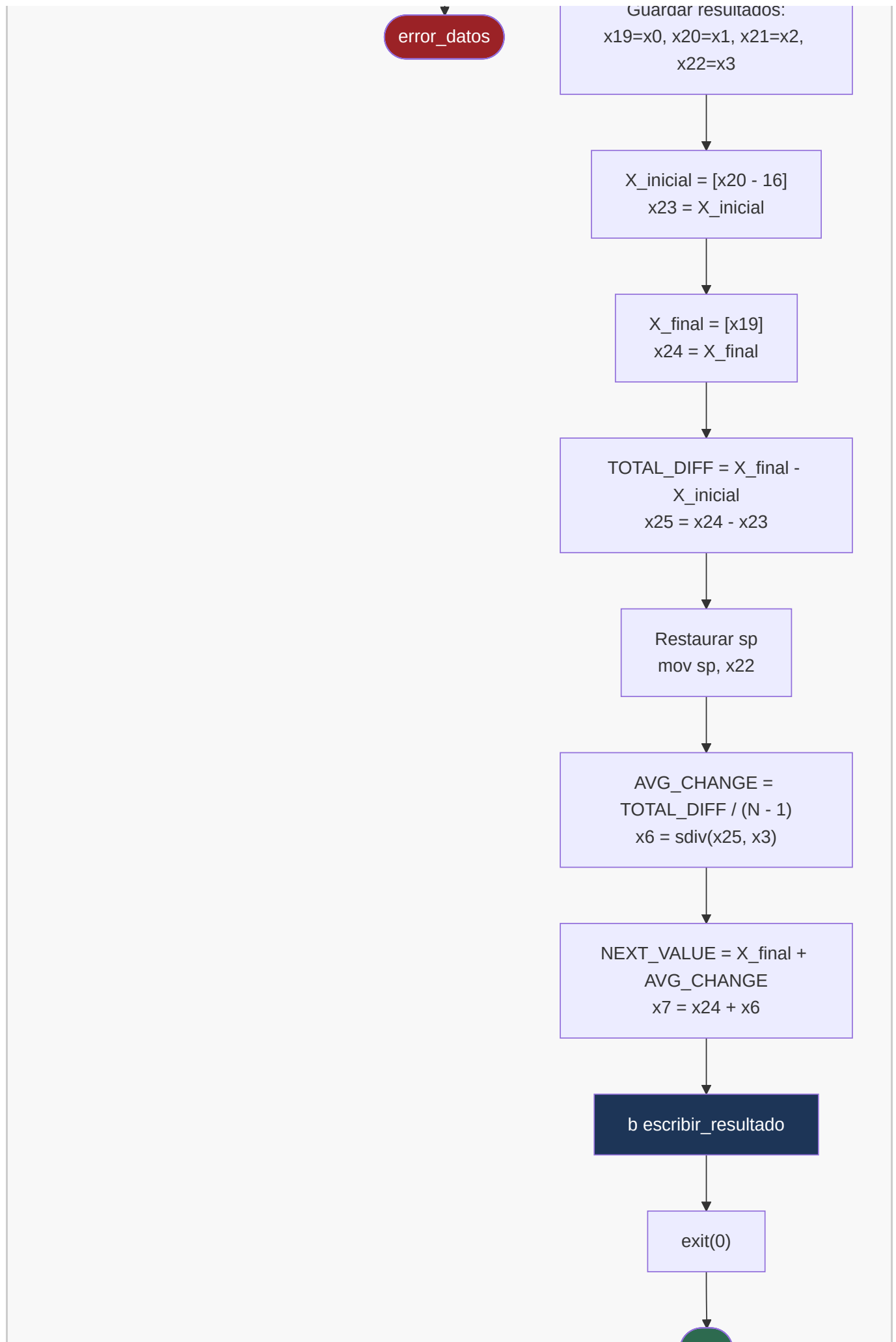
$$\text{PREDICCIÓN} = X_{\text{final}} + \text{PROMEDIO_CAMBIO}$$

El nombre del archivo a leer, la línea inicial a leer, la línea final y el número de columna a analizar se reciben como argumentos (`argv[1]` - `argv[4]`). El módulo delega en `utils.s` la lectura del CSV y la extracción de los N valores de esa columna, realiza los cálculos y escribe el resultado en `resultado_prediccion.txt`.

2. Diagrama de flujo

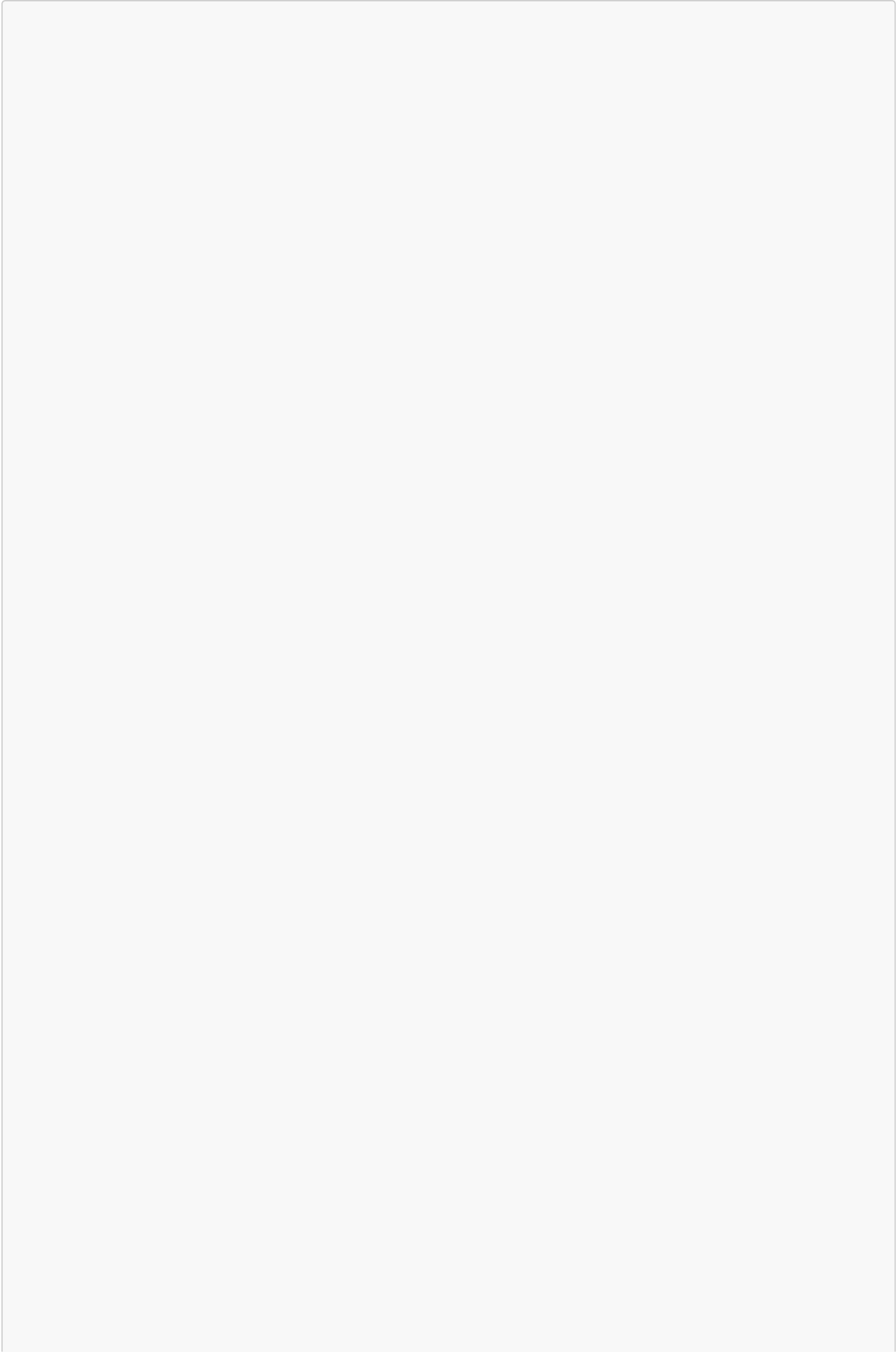








Subrutina **escribir_resultado**



Inicio escribir_resultado

x26 = dirección de
buffer_resultado

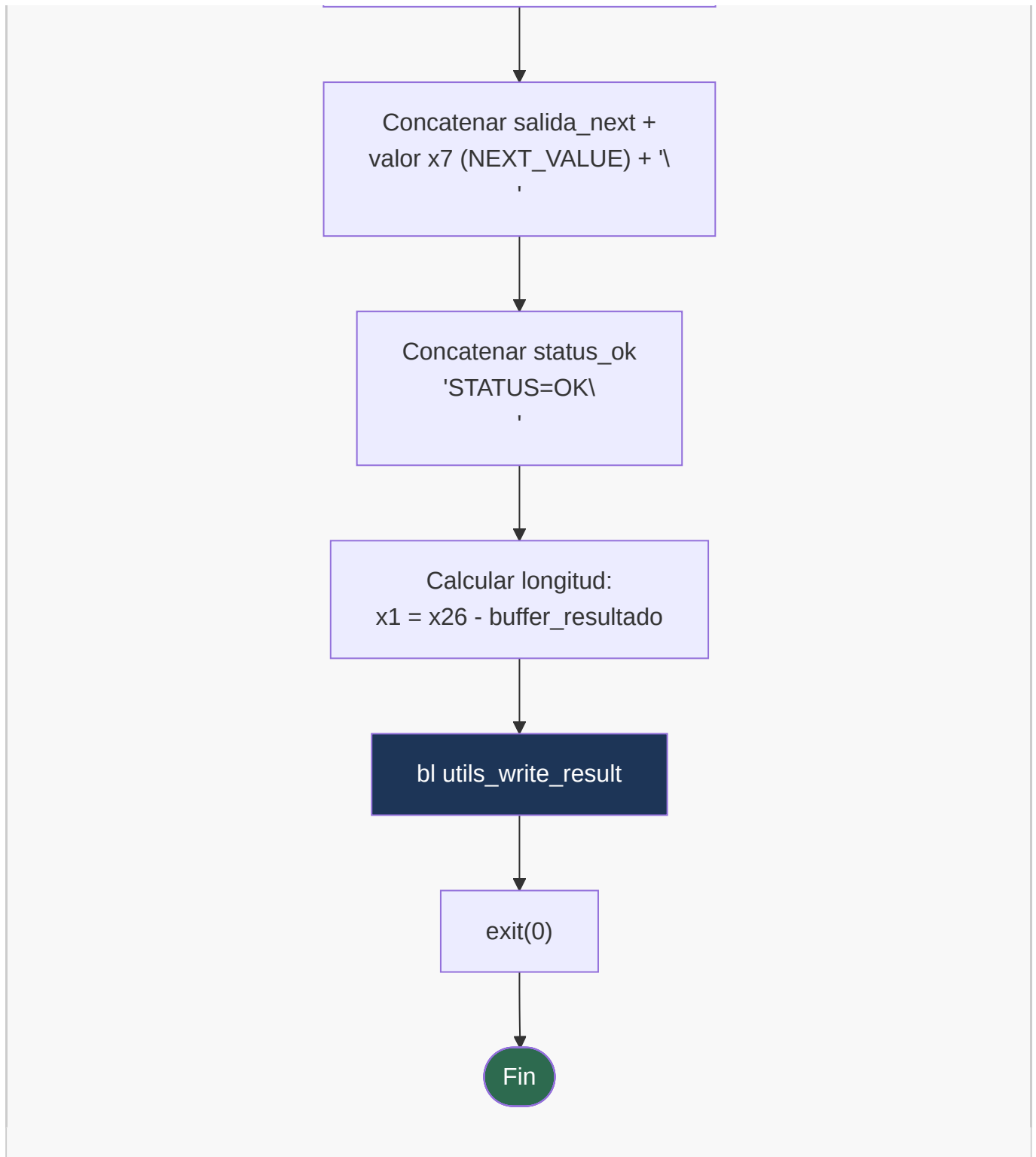
Concatenar salida_calculo
'CALC=PREDICTION\
,

Concatenar
salida_columna +
valor x27 + '\
,

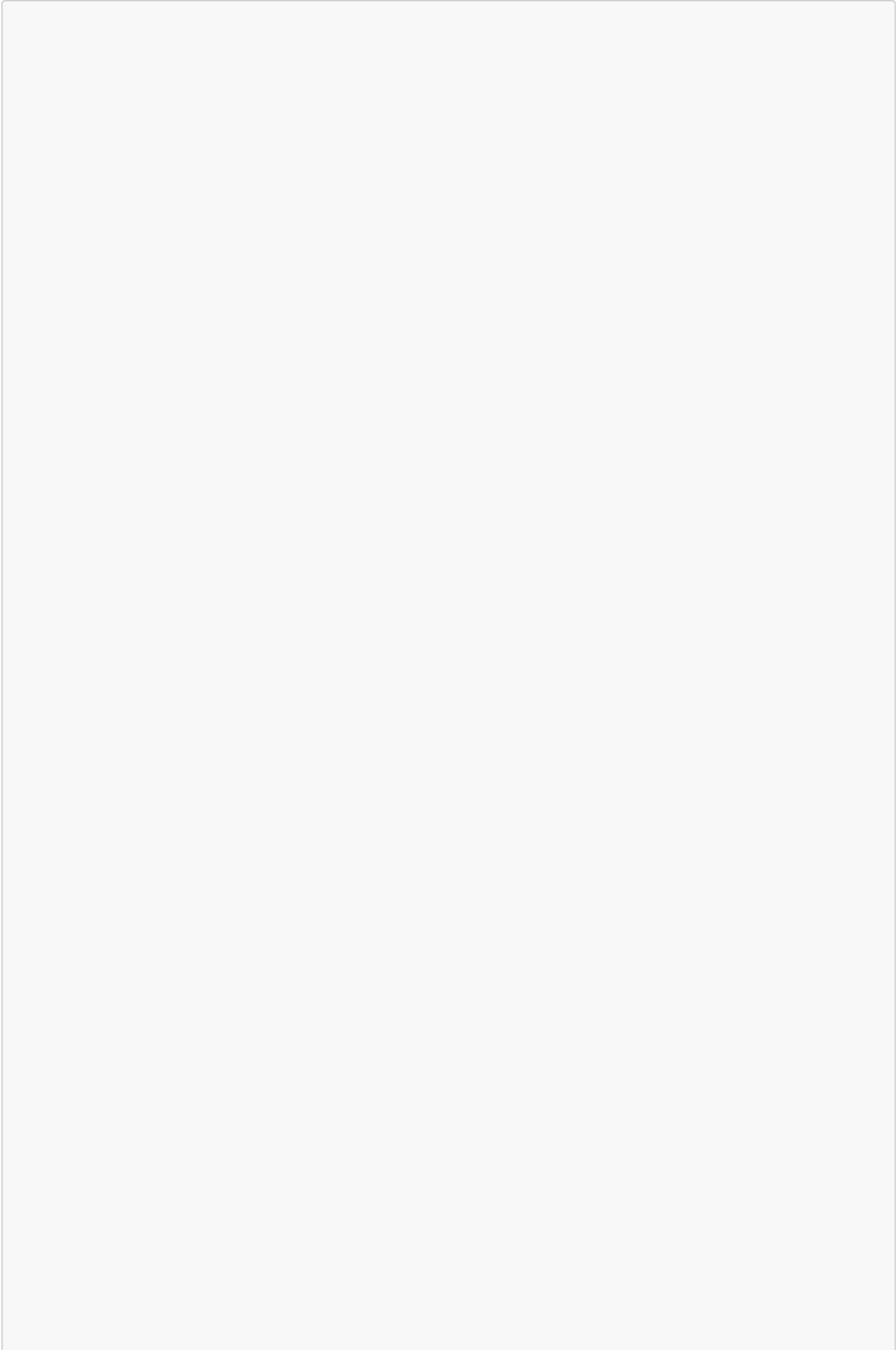
Concatenar salida_inicio +
valor x24
(WINDOW_START) + '\
,

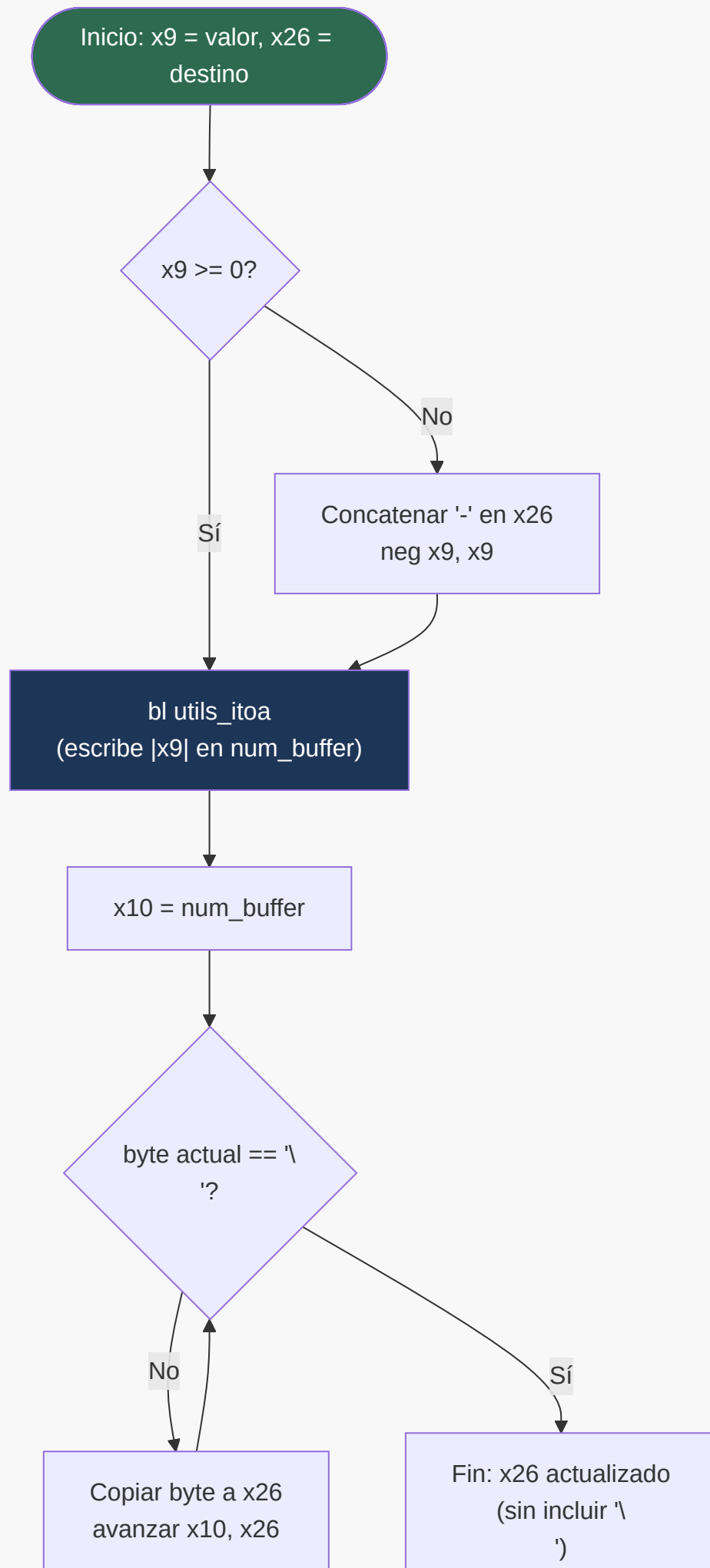
Concatenar salida_final +
valor x25 (WINDOW_END)
+ '\
,

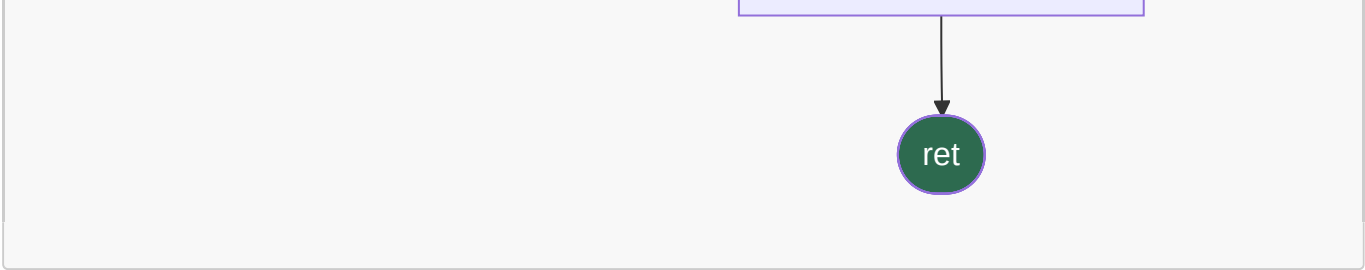
Concatenar
salida_contador +
valor x21 (COUNT) + '\
,

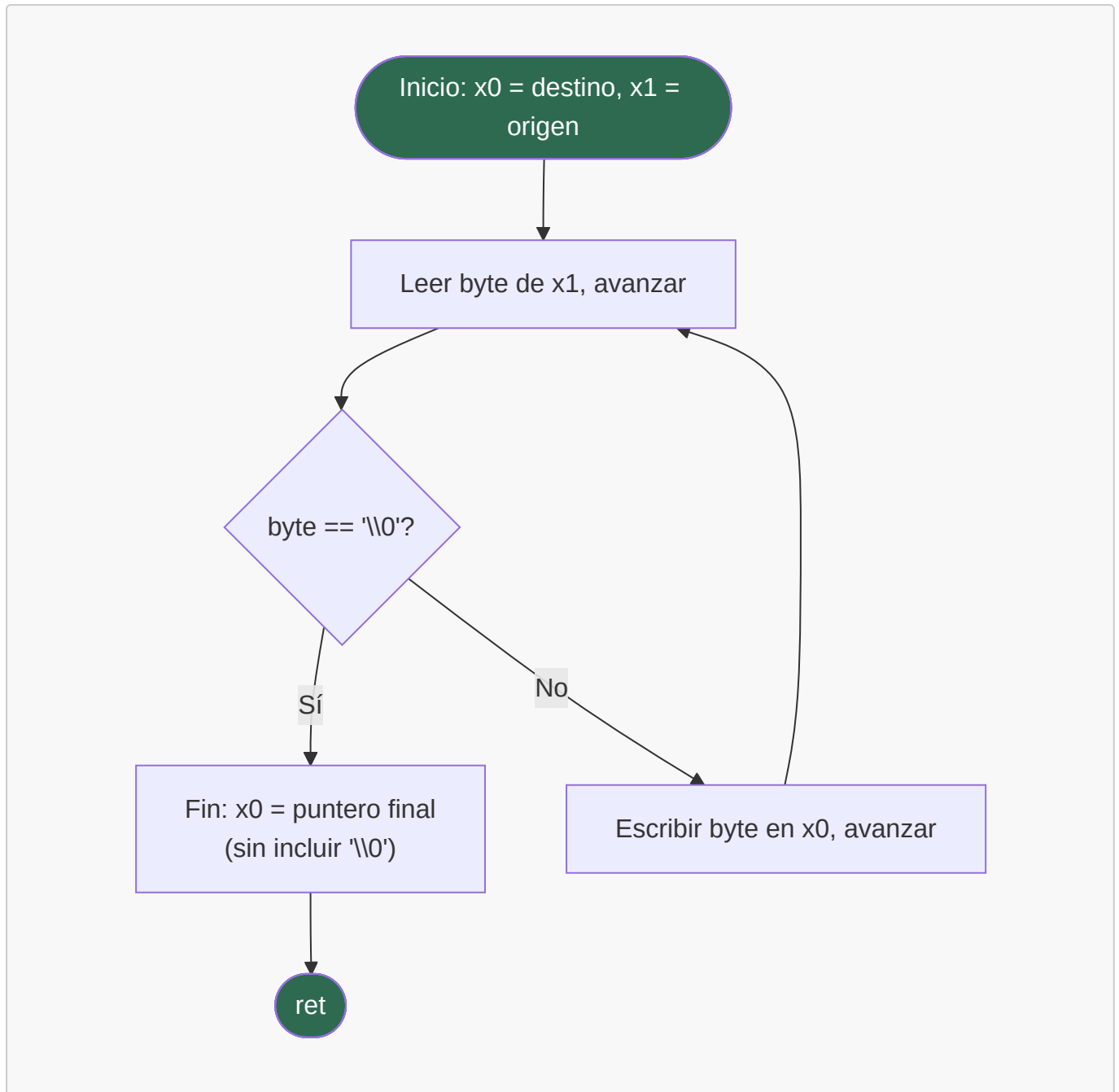


Subrutina `utilidad_escribir_entero`







Subrutina `utilidad_concatenar_cadena`

3. Explicación de los registros utilizados

Registro	Propósito
x11	Número de columna a leer (parámetro de entrada para <code>utils_read_column_to_stack</code>), obtenido de <code>argv[4]</code> .
x19	Dirección más baja del área de datos en el stack devuelta por <code>utils.s</code> (x0). Por el orden en que <code>utils_save_number</code> apila los valores, aquí queda el último dato leído del CSV → X_final.
x20	Dirección más alta del área de datos (x1/x28 de <code>utils.s</code>). x20 - 16 apunta al primer dato leído del CSV → X_inicial.
x21	Primero almacena el puntero al nombre del archivo (<code>argv[1]</code>); tras el retorno de <code>utils_read_column_to_stack</code> se sobrescribe con la cantidad de datos extraídos (N).
x22	Valor de <code>sp</code> original a restaurar al finalizar (x3 de <code>utils.s</code>), para dejar la pila balanceada.
x23	X_inicial (primer dato del CSV).
x24	Primero almacena la línea inicial (<code>argv[2]</code>); tras leer el stack se sobrescribe con X_final (último dato del CSV).
x25	Primero almacena la línea final (<code>argv[3]</code>); tras leer el stack se sobrescribe con $DIF = X_final - X_inicial$.
x27	Número de columna leída (sensor), guardado desde x11 antes de que <code>utils_read_column_to_stack</code> pueda modificarlo, para usarlo en la sección de escritura de resultados.
x6	$PROMEDIO_CAMBIO = DIF / (N - 1)$.
x7	$PREDICCIÓN = X_final + PROMEDIO_CAMBIO$.
x26	Puntero dinámico que recorre <code>buffer_resultado</code> mientras se construye el reporte de texto; se actualiza después de cada concatenación.
x9	Valor entero (con signo) que se pasa a <code>utilidad_escribir_entero</code> para convertir y escribir.
x0-x2, x10, x12	Registros de propósito general usados dentro de las subrutinas auxiliares para punteros, contadores y caracteres temporales (no preservados entre llamadas, de "uso libre" según convención).

4. Explicación de ciclos, saltos y subrutinas

Validación de argumentos

Lo primero que hace `_start` es comprobar que `argc == 5` (programa + archivo + inicio + fin + columna). Si no se cumple, salta a `error_argumentos` antes de leer cualquier argumento:


```
ldr x0, [sp]
cmp x0, #5
bne error_argumentos
```

Lectura de argumentos y conversión ASCII a entero

Se leen `argv[1]` (puntero al nombre del archivo), `argv[2]` (línea inicial) y `argv[3]` (línea final) mediante `atoi_argv`. El resultado numérico queda en `x10` y se copia a `x24` y `x25` respectivamente. La columna (`argv[4]`) se guarda en `x11`, que es el registro que `utils_read_column_to_stack` espera como parámetro de columna.

bl utils_read_column_to_stack (subrutina externa)

Llamada única que delega en `utils.s` toda la apertura y lectura del CSV y el parsing de la columna solicitada. Devuelve en `x0-x3` los punteros al área de datos en el stack, y en `x4` un código de error (1 = columna inválida, 2 = rango inválido). Las comprobaciones de error se realizan inmediatamente tras el retorno, antes de usar cualquier valor devuelto.

Cálculo de X_inicial y X_final (sin ciclo)

No hay ciclo porque solo se necesitan los extremos de la serie. El stack crece hacia abajo, por lo que el primer dato leído queda en la dirección más alta y el último en la más baja:

```
sub x0, x20, #16      // dirección de X_inicial
ldr x23, [x0]
ldr x24, [x19]        // X_final
```

escribir_resultado (secuencia lineal sin ciclos propios)

Construye el reporte concatenando en orden los bloques del formato de salida. Cada bloque sigue el mismo patrón:

```
mov x0, x26
ldr x1, =etiqueta
bl utilidad_concatenar_cadena    // copia la etiqueta, devuelve nueva
posición
mov x26, x0
mov x9, <registro_con_valor>
bl utilidad_escribir_entero      // escribe el número, actualiza x26
mov x0, x26
ldr x1, =salto_linea
bl utilidad_concatenar_cadena    // agrega '\n'
mov x26, x0
```

Al final calcula la longitud total (`x26 - dirección_inicial_del_buffer`) y llama a `utils_write_result` para escribir el archivo.

utilidad_escribir_entero (subrutina con ciclo de copia)

Maneja el signo manualmente porque `utils_itoa` solo soporta enteros positivos:

```
cmp x9, #0
bge .ue_positivo      // salto condicional: si x9 >= 0, omitir el signo
... escribir '-' y neg x9, x9 ...
.ue_positivo:
    bl utils_itoa      // convierte |x9| a "digitos\n\0" en num_buffer
    ldr x10, =num_buffer
.ue_copy:
    ldrb w12, [x10], #1 // ciclo: copia carácter por carácter
    cmp w12, #10       // ¿es '\n'?
    beq .ue_fin        // salto condicional: termina sin copiar el '\n'
    strb w12, [x26], #1
    b .ue_copy         // salto incondicional: repetir
.ue_fin:
```

El ciclo `.ue_copy` recorre el resultado de `utils_itoa` y lo transfiere al buffer de salida, **deteniéndose antes del `\n`** que `utils_itoa` siempre agrega, porque el formato `CLAVE=VALOR\n` ya añade su propio salto de línea por separado.

utilidad_concatenar_cadena (subrutina con ciclo de copia)

```
.loop_copia_bytes:
    ldrb w2, [x1], #1      // lee byte de origen, avanza
    cbz w2, .fin_copia_bytes // salto condicional: si es '\0', termina
    strb w2, [x0], #1      // escribe byte en destino, avanza
    b .loop_copia_bytes    // salto incondicional: repetir
.fin_copia_bytes:
    ret
```

Ciclo clásico de copia de string terminado en `\0`, usado para transferir las etiquetas estáticas y los caracteres especiales al buffer dinámico de salida.

Bloque de errores

Cada etiqueta de error (`error_argumentos`, `error_columna`, `error_rango`, `error_datos`) sigue la misma estructura: inicializar `x26` con la dirección de `buffer_resultado`, concatenar `STATUS=ERROR`, el campo `ERROR=` con su valor, el campo `DETAIL=` con su detalle, calcular longitud, escribir a `errores_prediccion.txt` y saltar a `exit_error`.

5. Manejo de errores

Error	Condición	Archivo de salida
INSUFFICIENT_ARGUMENTS	argc != 5	errores_prediccion.txt
INVALID_COLUMN	x4 == 1 tras utils_read_column_to_stack	errores_prediccion.txt
INVALID_RANGE	x4 == 2 tras utils_read_column_to_stack	errores_prediccion.txt
INSUFFICIENT_DATA	x2 < 2 (menos de 2 valores leídos)	errores_prediccion.txt

El formato de salida de error es:

```
STATUS=ERROR
ERROR=<TIPO_ERROR>
DETAIL=<DETALLE>
```

6. Explicación del formato de entrada y salida

Entrada

- **Argumento `argv[1]`:** nombre del archivo a leer.
- **Argumento `argv[2]`:** línea inicial del rango a leer.
- **Argumento `argv[3]`:** línea final del rango a leer.
- **Argumento `argv[4]`:** número de columna del sensor a analizar (2–7).
- **Formato del CSV:** encabezado seguido de filas de 9 columnas separadas por comas, terminado en \$:

```
ID, TEMP, HUM_AIRE, HUM_SUELO_1, HUM_SUELO_2, LUZ, GAS, RIEGO_1, RIEGO_2
1, 31, 65, 60, 18, 527, 50, 0, 0
...
30, 27, 74, 58, 15, 347, 91, 0, 0
$
```

Salida

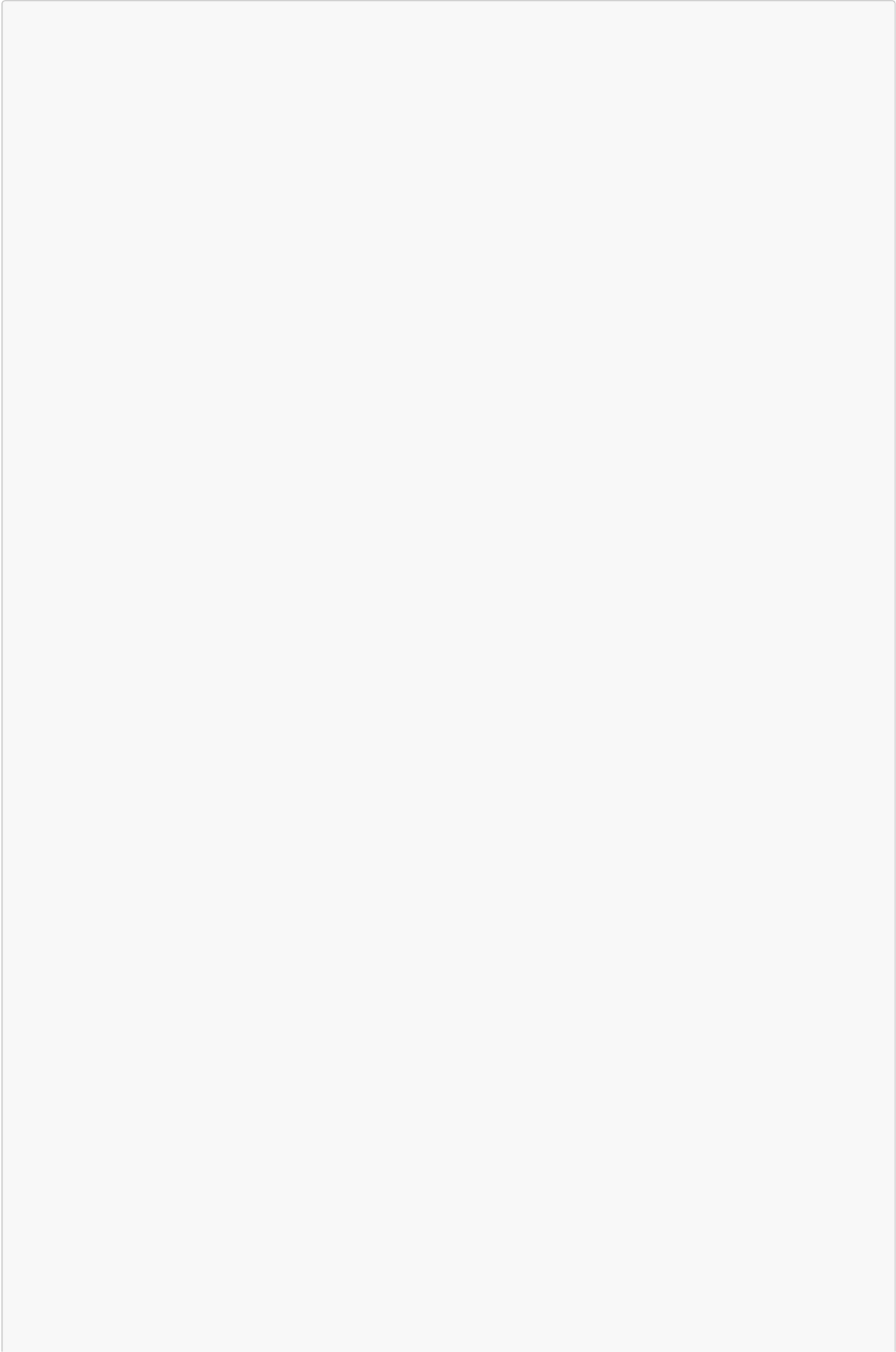
- **Archivo:** `resultado_prediccion.txt`, texto plano en formato `CLAVE=VALOR`, una por línea, leído posteriormente por el backend para almacenarse en MongoDB y mostrarse en el dashboard.
- **Formato de salida exitosa:**

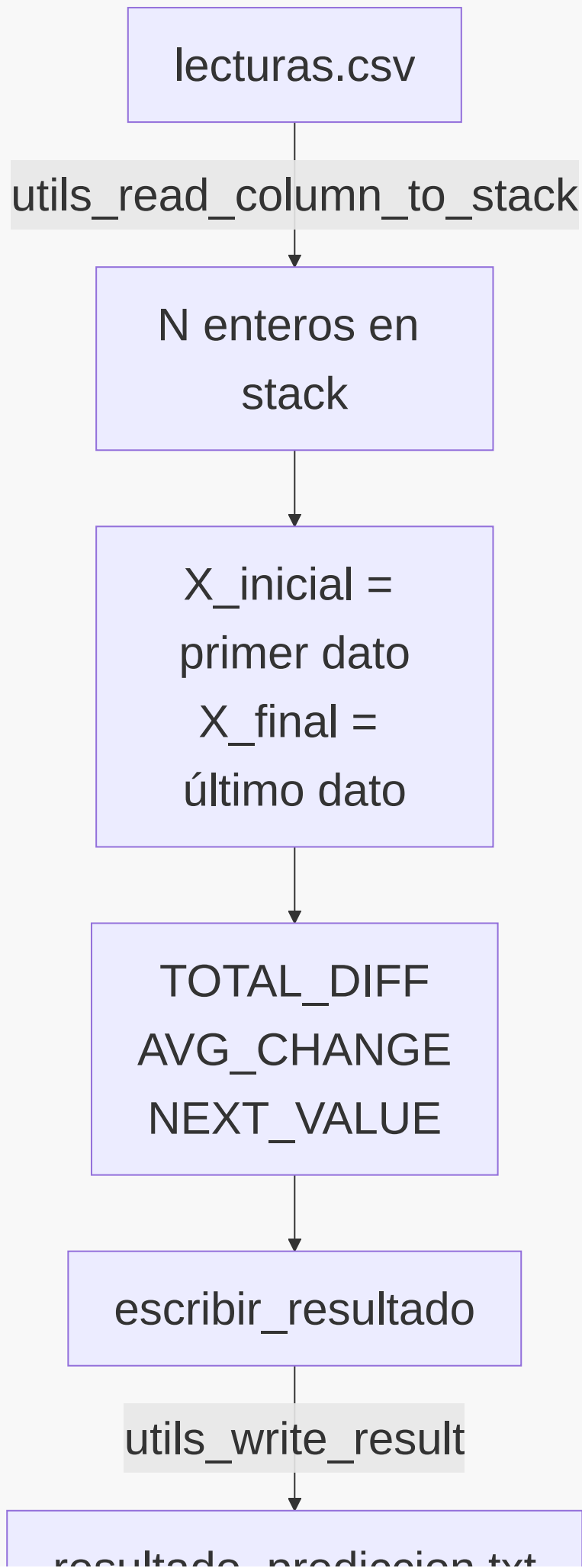
```
CALC=PREDICTION
COLUMN=<número_de_columna>
WINDOW_START=<línea_inicial>
WINDOW_END=<línea_final>
```

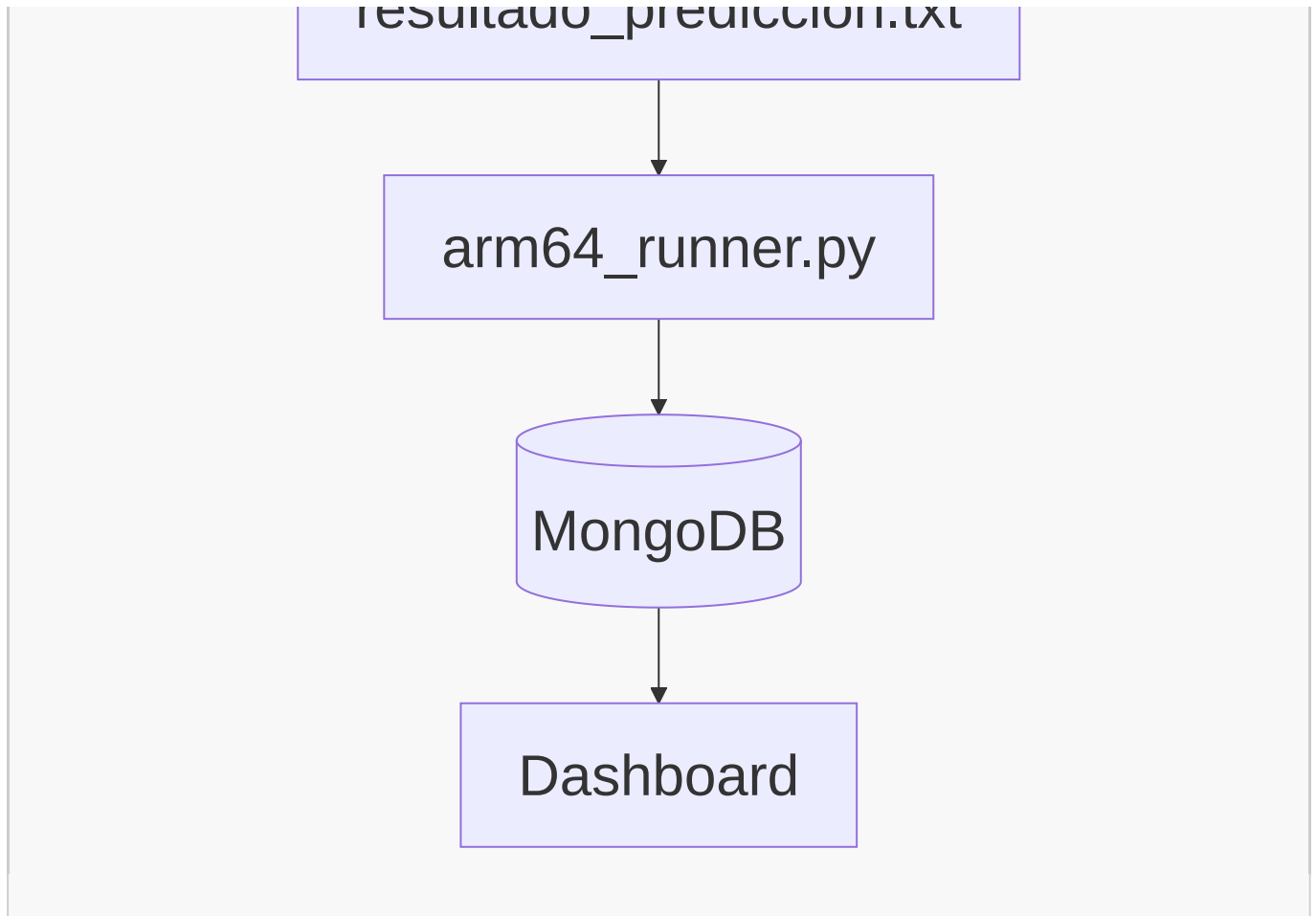
```
COUNT=<cantidad_de_datos>  
NEXT_VALUE=<predicción>  
STATUS=OK
```

- Todos los valores son **enteros con signo**; los valores negativos se representan con un - antepuesto.

7. Flujo general resumido







8. Nota sobre manejo de memoria

El módulo no reserva memoria propia para los datos: reutiliza directamente el área del stack que `utils_read_column_to_stack` deja apilada (`x19/x20`), y restaura `sp` (`mov sp, x22`) en cuanto extrae `X_inicial` y `X_final`, dejando la pila balanceada antes de construir el reporte.

La única memoria propia declarada está en `.bss`:

Buffer	Tamaño	Uso
<code>num_buffer</code>	32 bytes	Conversión temporal de enteros a texto (<code>utils_itoa</code>)
<code>buffer_resultado</code>	512 bytes	Reporte completo antes de escribirlo a disco

9. Depuración GDB

Se estaba comprobando el correcto funcionamiento de la lógica del programa y el flujo adecuado de la generación de archivos (de error en este caso)

